Event A

Baseline JDK

Our challenge for Event A was to select new projects from the original dataset that failed on the default JDK and were not part of our M2 testing. However, after checking the researchers' code and Dockerfile, we found that there was no specific default JDK defined.

Because of this, we decided to use JDK 17 as our baseline since we had already used it during M2 and could continue using the same Docker setup and scripts.

We will select projects that failed on JDK 17 and test them using JDK 11, 17, 21 and 23 to see if switching Java versions can fix the build failures.

We will check the Java version first, and if the build still fails, we will look into possible issues with dependencies, plugins or project configuration. This will help us understand which failures are actually related to the JDK version.

Project Selection and Prediction

For Event A, we selected five new projects from the researchers' original dataset that failed on our chosen baseline, JDK 17, and were not included in our M2 testing. We sorted the eligible projects alphabetically and selected the first five.

Our prediction is that changing the JDK version may fix some of the build failures, but not all of them. We expect that some projects may still fail because of dependencies, plugins or configuration issues.

We will test all five projects using JDK 11, 17, 21 and 23, following the same method we used during M2.

Selected Projects and their original Commits

1. AlmostReliable/lootjs d774ec2d319aca1576a3c925dbfaf75fb270c9dd

2. Argent-Matter/gcyr 2095a43abc41a34074350dba50fb8b381d2dd0a7

3. AutismSuperman/ArmorAuth 30f8f3b504faff15b8e25ba59218972037e7120e

4. BBS-Engine/bbs b3efac563acc9543fb6fbe9660258b1408af7950

5. Bamdoliro/marubase f45831ee4e138b9680d16520af876836a40a47ef

Results

Project	JDK 11	JDK 17	JDK 21	JDK 23
AlmostReliable/lootjs	FAIL	FAIL	FAIL	FAIL
Argent-Matter/gcyr	FAIL	FAIL	FAIL	FAIL
AutismSuperman/ArmorAuth	FAIL	FAIL	FAIL	FAIL
BBS-Engine/bbs	FAIL	FAIL	PASS	PASS
Bamdoliro/marubase	FAIL	FAIL	FAIL	FAIL

Project 1: AlmostReliable/lootjs

Default JDK (JDK 17): The build failed because the Gradle dependency almostgradle:1.1.1 requires Java 21 or higher.

JDK 11: This also failed because the same dependency requires Java 21 or higher. We didn't need to investigate it further because we had already identified the minimum Java version requirement.

JDK 21: The Java version issue was resolved, but the build still failed because Gradle could not find the required kubejs-neoforge:2101.7.0-build.131 dependency.

JDK 23: This failed because of the same missing dependency found on JDK 21.

Conclusion: Even though the project required Java 21 or higher, switching JDK versions did not fully fix the build. JDK 21 and 23 got past the Java version issue, but the project still failed because of a missing dependency. This shows that some build failures involve both Java compatibility and dependency management.

Project 2: Argent-Matter/gcyr

Default JDK (JDK 17): The build failed because Gradle could not find the required kubejs-forge:2001.6.4-build.120 dependency.

JDK 11: The build failed because the datafixerupper:6.0.8 dependency requires Java 17 or higher.

JDK 21: The build failed because of the same missing kubejs-forge dependency found on JDK 17.

JDK 23: The build failed with an Unsupported class file major version 67 error, suggesting that the project's build tools do not support Java 23.

Conclusion: Switching JDK versions did not fix this project. JDK 11 was too old for one of its dependencies, JDK 17 and 21 failed because of a missing dependency, and JDK 23 encountered a build-tool compatibility issue.

Project 3: AutismSuperman/ArmorAuth

Default JDK (JDK 17): The build failed because the frontend couldn't find a file called index.vue. This seems to be a missing file or project setup issue rather than a Java version problem.

JDK 11: The build failed because the project is set up to use Java 17 or higher.

JDK 21: The build failed because of an error involving the Java compiler. This suggests that one of the project's compilation tools may not work properly with JDK 21.

JDK 23: The build failed because the compiler couldn't find some symbols in the project's code. We couldn't confirm the exact reason from the errors we checked.

Conclusion: Switching JDK versions didn't fix the build. Although JDK 17 was suitable for the project's Java requirement, it still failed because of a missing frontend file. The newer JDKs also had compilation problems

Project 4: BBS-Engine/bbs

Default JDK (JDK 17): The build failed because the Java compiler was using US-ASCII encoding, which couldn't read some special characters in the project's source code. This caused several compilation errors.

JDK 11: The build failed because of the same encoding problem found on JDK 17.

JDK 21: The build passed successfully without changing the project's code.

JDK 23: The build also passed successfully without any code changes.

Conclusion: Switching from JDK 17 to JDK 21 or 23 successfully fixed the build in our tests. The failures on older JDKs were because of character encoding.

Project 5: Bamdoliro/marubase

Default JDK (JDK 17): The build failed because Gradle couldn't find an Excel template (.xlsx) file needed by the project.

JDK 11: The build failed because the Spring Boot Gradle plugin requires Java 17 or higher.

JDK 21: The build failed because the project's Gradle version (7.6.1) is not compatible with running on Java 21.

JDK 23: The build failed because the same Gradle version also does not support running on Java 23.

Conclusion: Switching JDK versions did not fix the build. JDK 11 was too old, JDK 17 had a missing file problem, and JDK 21 and 23 had Gradle compatibility issues.

M2 vs Event A Comparison

What transfers: Our M2 testing method worked on the new Event A projects without needing any changes. We were able to reproduce the original study's results using the same method.

What does not transfer: We did not identify any major finding from M2 that failed to transfer to Event A.

Strongest evidence: In Event A, we completed 20 builds and all of them gave the same results as the original study, just like the 80 builds we completed in M2.

Maintainability implication

Our results show that keeping build tools, dependencies and project configurations up to date is important. For example, Bamdoliro/marubase could not build on JDK 21 or 23 because its Gradle version was too old. Maintaining these tools properly could make projects easier to build across different Java versions.

Conclusion

For Event A, we tested five new projects using JDK 11, 17, 21 and 23, with JDK 17 as our default. All five projects failed on JDK 17, but one project, BBS-Engine/bbs, successfully built when we switched to JDK 21 and 23. The other four projects failed on all four JDK versions.

From investigating the build logs, we found that some failures were caused by Java version requirements, while others involved missing dependencies, missing files, encoding problems or incompatible build tools.

Overall, our results show that switching JDK versions can fix some build failures, but it does not work for every project. Sometimes the project also needs changes to its dependencies, files or build configuration to build successfully.